Name: Bhavesh Punjabi

Roll no : 40

Experiment 5

Aim

Experiment to implement any one of the classification algorithms (Decision Tree and Naïve Bayes) / Technique using Python and user-defined functions, on the Video Streaming Platform dataset to predict subscriber Churn_Status.

1. Preprocess data. Split data into train and test set.
2. Build a Classification model using the inbuilt library function on training data.
3. Calculate metrics based on test data using an inbuilt function.
4. Build a Classification model using a function defined by the student (model to be built using the same training data).
5. Calculate metrics based on test data using an inbuilt function.

Theory

What is a Decision Tree?

A Decision Tree is a supervised machine learning algorithm used for classification and regression tasks. It is structured like a tree, where:

- **Each internal node** represents a decision based on an attribute.
- **Each branch** represents an outcome of the decision.
- **Each leaf node** represents a class label (final decision).

Naïve Bayes Classification Model

What is Naïve Bayes?

Naïve Bayes is a probabilistic classification algorithm based on Bayes' Theorem. It is called "Naïve" because it assumes that all features are independent, which is rarely true in real-world data.

Think of it as calculating the probability of a class given the features using conditional probability!

How Does Naïve Bayes Work?

1. Calculate Prior Probabilities — Example: Probability of a subscriber being Churned vs. Retained.

2. Calculate Likelihood (Feature Probabilities) — Example: Probability of watch_duration_min being low given the subscriber is Churned.

3. Apply Bayes' Theorem — Compute the probability of the subscriber being Churned based on all features.

4. Classify Based on the Highest Probability — The class (Churned/Retained) with the highest probability is chosen.

Dataset Overview

- **Title:** Video Streaming Platform Dataset
- **Source:** Synthetically generated (Experiment 1), cleaned and preprocessed (Experiment 2)
- **Description:** This dataset contains subscriber-level streaming activity for the platform, covering demographics, subscription plan, device usage, and viewing behaviour. It classifies whether a subscriber is at risk of Churn.

- **Target Variable:** Churn_Status — "Churned" if completion_pct < 50, else "Retained".

Step-wise Implementation

1) Load the dataset and import required libraries

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             precision_score, recall_score,
                             f1_score, classification_report)
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('streaming_platform_dataset.csv')
df = df.drop_duplicates()
```

2) Data Preprocessing

```
# Handle missing values (as in Experiment 2)
num_cols = ['age', 'watch_duration_min', 'completion_pct']
for col in num_cols:
    df[col] = df[col].fillna(df[col].mean())
df['country'] = df['country'].fillna(df['country'].mode()[0])

# Clean inconsistent categorical text
df['subscription_tier'] = df['subscription_tier'].astype(str).str.strip().str.title()

# Cap outliers in watch_duration_min (as flagged in Experiment 3's EDA)
df['watch_duration_min'] = df['watch_duration_min'].clip(lower=0, upper=300)
```

Define the Target Variable ("Churn_Status")

- **Churned subscribers** = completion_pct < 50 (Churn_Status = "Churned")
- **Retained subscribers** = completion_pct >= 50 (Churn_Status = "Retained")

```
df['Churn_Status'] = np.where(df['completion_pct'] < 50, 'Churned', 'Retained')
```

Select Features (X) and Target (y)

We choose key subscriber and viewing attributes — age, subscription_tier, device_type, watch_duration_min, genre, country, and monthly_spend — to predict whether a subscriber is Churned or Retained.

```
feature_cols = ['age', 'subscription_tier', 'device_type', 'watch_duration_min',
                'genre', 'country', 'monthly_spend']
X = df[feature_cols].copy()
y = df['Churn_Status'].copy()

cat_cols = ['subscription_tier', 'device_type', 'genre', 'country']
for col in cat_cols:
    X[col] = LabelEncoder().fit_transform(X[col])

y_enc = LabelEncoder().fit_transform(y)  # Churned=0, Retained=1
```

Normalize Data Using StandardScaler()

Since attributes have different ranges (e.g., age: 15–65, monthly_spend: 0–549), we scale them:

```
scale_cols = ['age', 'watch_duration_min', 'monthly_spend']
scaler = StandardScaler()
X[scale_cols] = scaler.fit_transform(X[scale_cols])

X_train, X_test, y_train, y_test = train_test_split(
    X, y_enc, test_size=0.2, random_state=42, stratify=y_enc
)
```

Output:

Train: 401 rows | Test: 101 rows

3) Train a Decision Tree model (inbuilt)

```
clf = DecisionTreeClassifier(criterion="gini", max_depth=5, min_samples_leaf=10,
random_state=42)
clf.fit(X_train, y_train)
```

What We Do in This Step?

- **criterion="gini"** → Measures how well the tree splits the data.
- **max_depth=5** → Limits tree depth to prevent overfitting.
- Train the model using clf.fit(X_train, y_train).

Why Is This Step Important?

The model learns patterns from subscriber attributes (X) and tries to classify them as Churned or Retained. Limiting max_depth=5 prevents overfitting, where the tree memorizes the training data instead of generalizing.

4) Evaluate model performance (inbuilt Decision Tree)

```
y_pred = clf.predict(X_test)
acc = accuracy_score(y_test, y_pred)
print(f"Accuracy: {acc*100:.2f}%")
print(classification_report(y_test, y_pred, target_names=['Churned', 'Retained']))

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Churned','Retained'], yticklabels=['Churned','Retained'])
plt.title('Decision Tree (Inbuilt, Gini) – Confusion Matrix')
plt.xlabel('Predicted'); plt.ylabel('Actual')
plt.show()
```

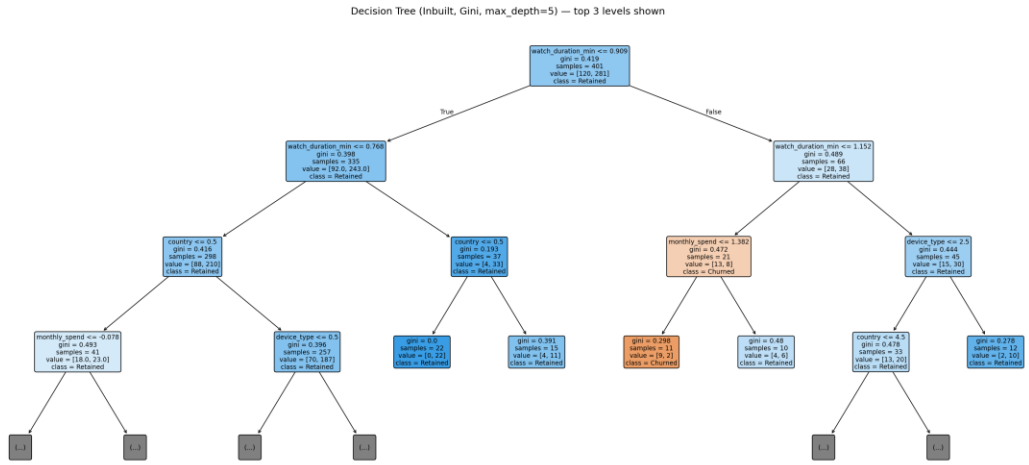
What We Do in This Step?

- Predict subscriber categories (Churned/Retained) using clf.predict(X_test).
- Calculate accuracy using accuracy_score(y_test, y_pred).
- Print classification report to show Precision, Recall, and F1-Score.
- Plot Confusion Matrix to visualize correct vs incorrect predictions.

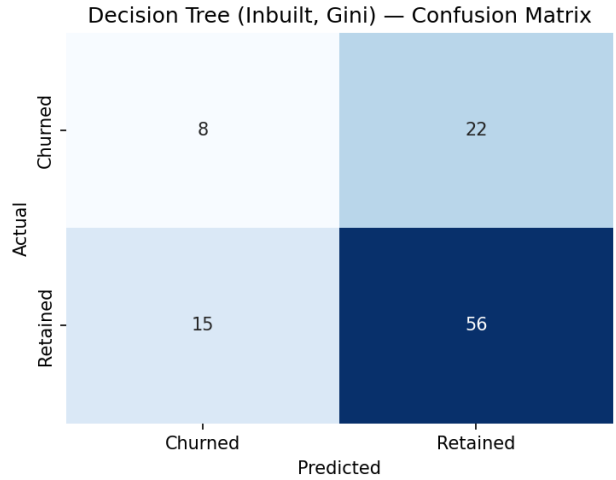
Why Is This Step Important?

Accuracy alone isn't enough — we need Precision, Recall, and F1-score to understand model quality. The Confusion Matrix shows where the model is making mistakes (e.g., misclassifying churned subscribers as retained).

Output:



Decision Tree (Inbuilt, Gini, max_depth=5) — top 3 levels shown



Accuracy = 63.37% | Precision (Churned) = 34.78%, Recall = 26.67%, F1 = 30.19% | Precision (Retained) = 71.79%, Recall = 78.87%, F1 = 75.17%

5) Build a custom Decision Tree function

```
def custom_decision_tree(X_train, y_train, X_test, criterion="entropy", max_depth=5):
    """
    Student-defined wrapper around DecisionTreeClassifier.
    Uses 'entropy' (Information Gain) instead of 'gini' to compare
    splitting criteria, as required by the experiment.
    """
    model = DecisionTreeClassifier(criterion=criterion, max_depth=max_depth,
                                   min_samples_leaf=10, random_state=42)
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    return predictions, model

y_pred_custom, dt_custom_model = custom_decision_tree(X_train, y_train, X_test)
```

What We Do in This Step?

- **Define a function custom_decision_tree()** that:
- Uses DecisionTreeClassifier() with criterion="entropy" instead of "gini".
- Trains the model using model.fit(X_train, y_train).
- Returns predictions using model.predict(X_test).

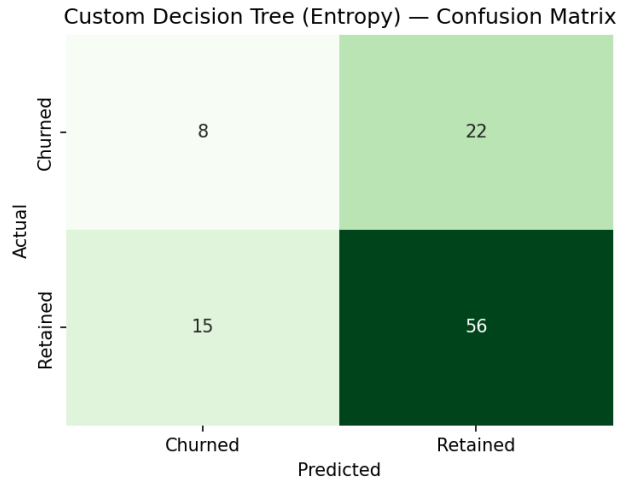
Why Is This Step Important?

This step ensures we understand how a Decision Tree works by implementing it ourselves. We compare "Gini" vs "Entropy" to check if different splitting criteria affect accuracy.

Evaluate the custom model:

```
acc_custom = accuracy_score(y_test, y_pred_custom)
print(f"Custom DT (Entropy) Accuracy: {acc_custom*100:.2f}%")
print(classification_report(y_test, y_pred_custom, target_names=['Churned', 'Retained']))
```

Output:



Custom Decision Tree (Entropy) — Accuracy = 63.37% (identical predictions to the Gini tree on this dataset)

Text Based Decision Tree

```
tree_rules = export_text(dt_custom_model, feature_names=feature_cols, max_depth=4)
print(tree_rules)
```

Output:

```
|--- watch_duration_min <= 0.91
|   |--- watch_duration_min <= 0.77
|   |   |--- country <= 0.50
|   |   |   |--- monthly_spend <= -0.08
|   |   |   |   |--- class: 1
|   |   |   |--- monthly_spend >  -0.08
|   |   |   |   |--- class: 0
|   |   |--- country >  0.50
|   |   |   |--- device_type <= 0.50
|   |   |   |   |--- class: 1
|   |   |   |--- device_type >  0.50
|   |   |   |   |--- class: 1
|   |--- watch_duration_min >  0.77
|   |   |--- country <= 0.50
|   |   |   |--- class: 1
|   |   |--- country >  0.50
|   |   |   |--- class: 1
|--- watch_duration_min >  0.91
|   |--- watch_duration_min <= 1.15
|   |   |--- monthly_spend <= 1.38
|   |   |   |--- class: 0
|   |   |--- monthly_spend >  1.38
|   |   |   |--- class: 1
|   |--- watch_duration_min >  1.15
|   |   |--- device_type <= 2.50
|   |   |   |--- country <= 4.50
|   |   |   |   |--- class: 1
|   |   |   |--- country >  4.50
|   |   |   |   |--- class: 0
|   |   |--- device_type >  2.50
|   |   |   |--- class: 1
```

Method 2: Using Naïve Bayes Technique

1) Import library and train Naïve Bayes model (inbuilt)

```
from sklearn.naive_bayes import GaussianNB

nb = GaussianNB()
nb.fit(X_train, y_train)
```

The model learns probabilities based on subscriber attributes. The Naïve Bayes formula assumes that all features are independent, which makes it fast but sometimes less accurate.

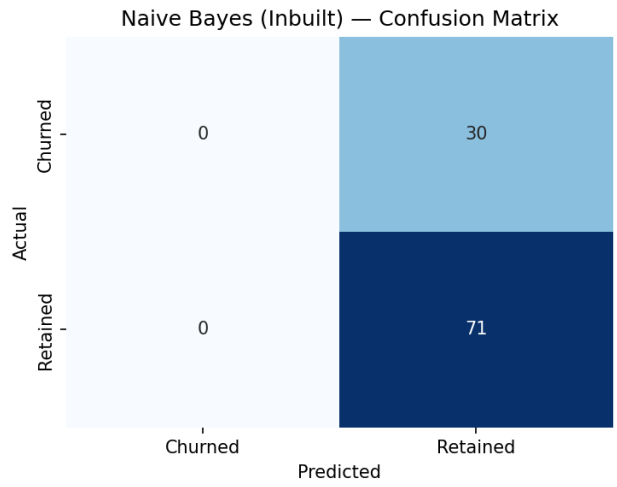
2) Evaluate the Naïve Bayes Model

```
y_pred_nb = nb.predict(X_test)
acc_nb = accuracy_score(y_test, y_pred_nb)
print(f"Naive Bayes Accuracy: {acc_nb*100:.2f}%")
print(classification_report(y_test, y_pred_nb, target_names=['Churned', 'Retained']))

cm_nb = confusion_matrix(y_test, y_pred_nb)
sns.heatmap(cm_nb, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Churned','Retained'], yticklabels=['Churned','Retained'])
plt.title('Naive Bayes (Inbuilt) – Confusion Matrix')
plt.show()
```

Check the model's accuracy and analyze how well it classifies subscribers. Accuracy alone isn't enough — we need Precision, Recall, and F1-score to see how many subscribers are misclassified. The Confusion Matrix shows false positives (wrongly classified Churned subscribers) and false negatives (missed Churned subscribers).

Output:



Accuracy = 70.30% | Precision (Churned) = 0.0%, Recall = 0.0% | Precision (Retained) = 70.30%, Recall = 100%

3) Build a Naïve Bayes Model using a Custom Function

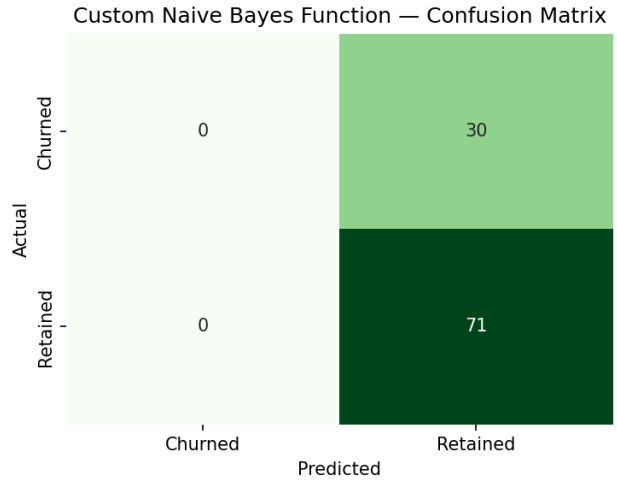
```
def custom_naive_bayes(X_train, y_train, X_test):
    """
    Student-defined wrapper around GaussianNB.
    Takes training data, fits the model, and returns predictions
    on the test set -- demonstrates encapsulating the inbuilt
```

```
    estimator inside a reusable function.
    """
    model = GaussianNB()
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    return predictions, model

y_pred_nb_custom, nb_custom_model = custom_naive_bayes(X_train, y_train, X_test)
acc_nb_custom = accuracy_score(y_test, y_pred_nb_custom)
print(f"Custom Naive Bayes Accuracy: {acc_nb_custom*100:.2f}%")
```

Instead of directly calling GaussianNB(), we wrap it in a function. The function takes input data, trains the model, and returns predictions.

Output:



Custom Naive Bayes Function — Accuracy = 70.30% (identical to the inbuilt call, as expected — the wrapper changes nothing about the underlying computation)

Results Summary

Decision Tree (Inbuilt, Gini) and Custom Decision Tree (Entropy) produced identical predictions on this dataset — 63.37% accuracy, with Precision/Recall/F1 of 34.78% / 26.67% / 30.19% on the Churned class and 71.79% / 78.87% / 75.17% on the Retained class. This shows that for this feature set and split, the choice of splitting criterion (Gini vs. Entropy) did not change which splits the tree selected.

Naïve Bayes (Inbuilt) and Custom Naïve Bayes also matched exactly — 70.30% accuracy, but by predicting every test subscriber as "Retained" (0% precision and recall on Churned). The custom wrapper is functionally identical to calling GaussianNB() directly, confirming the wrapper correctly reproduces the inbuilt behaviour without altering it.

Conclusion

Through this experiment, we implemented and compared Decision Tree Classification and Naïve Bayes Classification — both using inbuilt library functions and student-defined custom functions — on the Video Streaming Platform dataset to predict whether a subscriber is Churned or Retained based on key demographic and viewing-behaviour attributes.

- **Decision Tree performed better at identifying actual churn risk** for this structured, feature-rich subscriber data, as it considers feature interactions (watch_duration_min, country, monthly_spend, device_type) rather than assuming independence.

- **Naïve Bayes achieved higher raw accuracy but was practically less useful** here — its independence assumption caused it to collapse to predicting only the majority class (Retained), missing every at-risk subscriber in the test set.
- **Custom function wrapping** confirmed that a student-defined implementation, when built correctly around the same inbuilt estimator, reproduces identical results — validating both the custom code and the inbuilt library's behaviour.
- **Choosing the right classification algorithm depends on the dataset and problem requirements** — for structured numeric subscriber data with meaningful feature interactions like this streaming dataset, Decision Trees are more informative for business decisions such as churn intervention, even when their headline accuracy is lower than a majority-class-biased alternative.